

Notas Técnicas — Arquitetura, DevOps e Uso de IA

Complemento ao documento de escopo e ao documento de páginas e fluxos. Este arquivo reúne as decisões, recomendações e pontos em aberto discutidos sobre como o projeto vai ser construído (não sobre o que ele faz).

1. Arquitetura: linguagens e como o núcleo conversa com o OpenCL

O kernel de OpenCL em si é sempre escrito em **OpenCL C** — isso não muda, é a linguagem da especificação. A decisão real é qual linguagem hospeda esse kernel e o backend web:

C/C++ com a API oficial (cl.h) — mais controle, mais alinhado ao que se espera de rigor técnico em Tópicos Avançados, mas mais trabalho de baixo nível (parsing HTTP, ORM, etc. não vêm prontos).

Python + PyOpenCL — desenvolvimento mais rápido, boa integração com NumPy/Pillow/OpenCV para pré-processamento de imagem.

[decidido] A equipe optou por usar **C apenas no núcleo OpenCL** — é a opção mais natural (linguagem mais próxima do kernel, sem camada de tradução) e o que a disciplina de Tópicos Avançados provavelmente espera ver. C na stack inteira (incluindo a API web) foi descartado: implicaria abrir mão de frameworks web prontos e assumir manualmente parsing HTTP, banco de dados e autenticação — escopo de trabalho desproporcional ao prazo do projeto.

[decidido] O **backend web fica em Python**.

Recomendação de arquitetura, independente da linguagem escolhida: o núcleo OpenCL não deve morar dentro do backend web principal. Ele deve ser um **serviço separado**, com uma API enxuta (ex.: [POST /comparar {foto, filtros}] → [{candidatos ordenados}]). O backend web chama esse serviço via HTTP interno. Isso permite: (1) o time trabalhar no backend sem precisar do runtime OpenCL instalado; (2) otimizar/medir o núcleo isoladamente; (3) rodar o benchmark comparativo (RF-29) num ambiente controlado, sem ruído do resto da aplicação.

1.1 Stack proposta para o backend Python

FastAPI como framework — assíncrono nativamente (importante porque o backend fica chamando o serviço em C via HTTP o tempo todo, sem travar outras requisições enquanto espera resposta), gera documentação OpenAPI/Swagger automática (isso já entrega de graça o contrato de API que o time de frontend precisa consultar) e usa Pydantic para validar entrada, o que ajuda inclusive o Claude Code a gerar código mais confiável por causa da tipagem

SQLAlchemy (2.0) + Alembic — ORM e migrações de banco, falando com o Postgres/Supabase

httpx — cliente HTTP assíncrono do Python, usado pelo FastAPI para chamar o serviço do núcleo em C

JWT (via [python-jose] ou [fastapi-users]) — para o login único com emissão de token (RF-02)

RQ (Redis Queue) — fila para o processamento em lote do monitoramento contínuo (RF-25); mais simples de configurar que Celery para o tamanho desse projeto, e já usa o Redis que o Docker Compose já provisiona (seção 2). Celery é uma alternativa mais robusta se o time preferir, ao custo de mais configuração

pytest — testes automatizados, que entram direto no pipeline do GitHub Actions (seção 3)
ruff — lint e formatação num único comando, mais rápido de configurar no CI que combinar flake8 + black + isort separadamente

2. Papel do Docker

O problema real que o Docker resolve aqui não é "deploy" — é o **runtime de OpenCL**, que é instalado de forma diferente em Windows/Mac/Linux e costuma quebrar entre máquinas do time. Como o núcleo roda em **dispositivo CPU** (não GPU), a solução é instalar o **POCL** (Portable Computing Language, um runtime de OpenCL por software) dentro do container do serviço — não é preciso GPU nem container com GPU passthrough.

Composição sugerida para `docker-compose.yml` local:

`frontend` (dev server)

`api` (backend web)

`nucleo-opencl` (com POCL embutido)

`db` (Postgres local, espelhando o banco de produção)

`redis` (fila para o processamento em lote do monitoramento contínuo, RF-25 — via RQ/Celery em Python ou BullMQ em Node)

3. DevOps gratuito

[verificado em setembro/2026 – free tiers mudam com frequência, confirmar na página oficial antes de decidir]

Frontend

Vercel (Hobby): gratuito, 100 GB de banda, preview automática por Pull Request, uso não comercial (cobre projeto de faculdade)

Alternativa: Cloudflare Pages (banda ilimitada para estático)

Backend e núcleo OpenCL

Render (free): 750h de instância/mês, mas só 0,1 vCPU e 512 MB de RAM, e o serviço dorme por inatividade — serve para uma API de demonstração, **não é confiável para gerar a métrica de desempenho com/sem OpenCL**. Rodar os benchmarks localmente ou em laboratório da faculdade, e usar a hospedagem gratuita só para a demo ao vivo.

Evitar Railway para isso: o plano gratuito hoje é um trial de 30 dias com crédito, depois cai para uma cota muito pequena por mês — não dura um semestre.

Banco de dados

Supabase (free): Postgres com extensão PostGIS (bom para consultas de raio/distância do mapa), 500 MB de banco, pausa o projeto após 1 semana de inatividade (reativar antes de cada sprint/apresentação resolve).

CI/CD e versionamento

GitHub: repositório, Issues e Projects (kanban) gratuitos, colaboradores ilimitados

GitHub Actions: ilimitado em repositório público; 2.000 minutos/mês grátis em repositório privado (suficiente para um time pequeno)

Bônus

GitHub Student Developer Pack — continua ativo em 2026, libera créditos de nuvem, JetBrains, domínio grátis etc. via verificação de e-mail institucional (não inclui Copilot gratuito, pausado para novas inscrições desde abril/2026).

4. Métricas técnicas para o benchmark (RF-29 / seção 4.6 do escopo)

O argumento central do projeto para Tópicos Avançados é a curva de escala — vale medir mais do que só "ficou mais rápido":

Tempo de busca × tamanho da base ($N = 10, 100, 1.000, 10.000$ registros), com e sem OpenCL

Speedup (tempo sequencial ÷ tempo paralelo) em função de N

Breakdown por etapa do pipeline: pré-processamento, extração de descritores, comparação em massa, medidos separadamente (cada um paraleliza de forma diferente)

Throughput (comparações por segundo)

Escala 2D do lote de monitoramento (N buscas ativas × M novos registros/dia) — vale um heatmap ou gráfico de superfície, não só uma curva

Overhead de transferência host↔device — mesmo em CPU há custo de contexto/buffer no OpenCL; reportar isso mostra entendimento dos limites do modelo

Métrica de corretude, separada de performance: confirmar que a versão paralela devolve o mesmo ranking de similaridade que a versão sequencial

Referência teórica útil para justificar o teto de speedup: **Lei de Amdahl**, dado a fração não-paralelizável do pipeline

5. Estratégia de uso de IA / tokens

Ferramenta para codar: Claude Code é mais eficiente em tokens do que colar arquivos inteiros num chat — ele lê só os trechos do repositório que precisa, sob demanda.

Organização de contexto:

Um **CLAUDE.md** na raiz do repositório com arquitetura, stack e convenções — Claude Code lê isso automaticamente a cada sessão. Pode ser compartilhado via controle de versão (memória de projeto) e cada integrante ainda pode ter preferências pessoais que ficam só na própria máquina, sem afetar o repositório.

Manter esse arquivo curto e objetivo — um CLAUDE.md muito longo vira ruído competindo por espaço com o que importa na sessão

Uma conversa/sessão por componente ou por tarefa, não uma conversa única para o projeto inteiro

Ao trabalhar no núcleo OpenCL, mandar só o kernel/arquivo relevante, não o repositório inteiro

O documento de escopo e o de páginas/fluxos valem como referência fixa anexada a um Projeto do Claude, para sessões de UI não precisarem reexplicar regra de negócio do zero

6. Pontos em aberto trazidos nesta conversa

Nenhum pendente no momento — linguagens do núcleo (C) e do backend (Python/FastAPI) já estão decididas.